

Predicting Apartment Rental Prices By Machine Learning

Alican Akkiprik

Politecnico di Torino

Student id: s349610

s349610@studenti.polito.it

Emin Oral

Politecnico di Torino

Student id: s343124

s343124@studenti.polito.it

Abstract—The accurate prediction of apartment rental prices stands as a major difficulty in contemporary real estate analytics particularly in fast-changing urban real estate markets. The project developed a machine learning pipeline which integrates structured data elements (size, location and number of rooms) with unstructured information (text descriptions and amenities) to predict monthly rents. The team tested multiple regression algorithms before choosing LightGBM because of its exceptional performance and ability to scale. The final model achieved a Mean Absolute Error (MAE) of \$169.20 after performing extensive hyperparameter tuning and cross-validation on the validation set. The solution provides useful insights to stakeholders while establishing a strong framework for analyzing future rental market trends.

I. PROBLEM OVERVIEW

Determining or evaluating rates can be a biased task that lacks consistent benchmarks for comparison purposes, in a digital age where there is a wealth of data on apartment listings available for analysis, by machine learning algorithms to recognize trends [1] and make more precise predictions.

The projects dataset comprises to 100,000 apartment listings gathered from locations, throughout the United States with a mix of structured elements like square footage and bedroom count and unstructured components like descriptive text and amenity details. Our objective involved harnessing both data types to construct scalable models, for predicting prices effectively.

The task, at hand is presented as a regression challenge where the input characteristics are derived from every aspect and the desired outcome is the monthly rental cost in USD of model performance is assessed through the Mean Absolute Error (MAE). The primary goal is to establish a process that offers information for market researchers as well, as property owners and tenants through fostering clear and data oriented rental pricing systems.

II. PROPOSED APPROACH

A. Preprocessing

The construction of a reliable predictive model required us to develop a complete data preprocessing pipeline through scikit-learn [1] tools. Our steps included:

- The median imputation method was used to handle missing values in numerical fields square_feet and bathrooms. The missing values in categorical features pets_allowed and state were replaced with the most frequent category which is “mode”. Text fields were filled with empty strings to ensure proper vectorization.
- The target variable price received outlier removal through filtering which retained records between the 0.5th and 99.5th percentiles. The removal of rare anomalous data points through this method improved model robustness.



Figure 1: Distribution of monthly rental price

- The numerical features received standardization through zero mean and unit variance transformation to prevent any feature from dominating the model training process. The OneHotEncoding method was used to encode categorical variables so that all category information remained intact while avoiding any artificial ordering
- The unstructured data underwent feature engineering through TF-IDF vectorization of the “title”, “body”, and “amenities” columns. [8] The selection of TF-IDF features for each text field was based on exploratory analysis to extract descriptive details and essential resources.
- The model used geospatial binning to process latitude and longitude by multiplying and rounding these values so it could learn neighborhood effects without needing complex clustering.

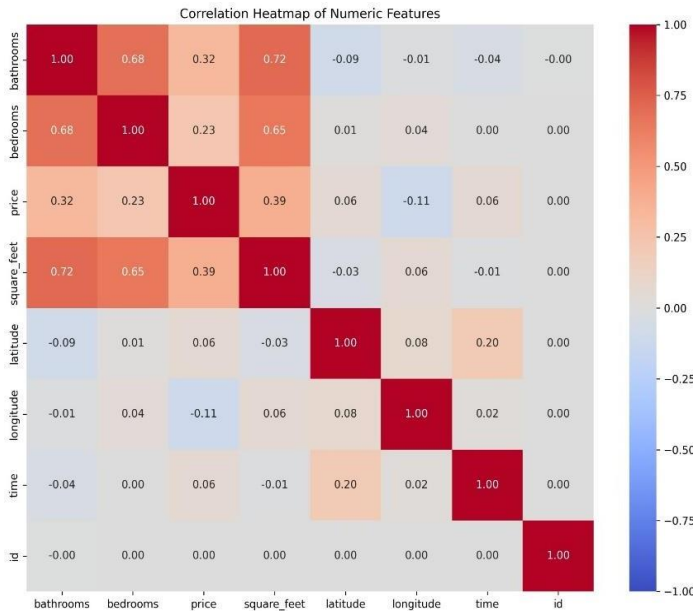


Figure 2: The correlation heatmap shows linear relationships between numeric features and the target which guides our feature selection and engineering strategy.

The preprocessing steps were developed as modular reusable pipelines with scikit-learn to maintain consistency between training, validation and test data. [1][9]

B. Model Selection

The initial experiments involved testing different regression models for rental price prediction. Which are:

- Linear Regression
- Gradient Boosting Regressor
- XGBoost [2]
- LightGBM (LGBMRegressor) [3]

We assessed each method using cross-validation and the Mean Absolute Error (MAE) metric according to project requirements. Linear Regression performed well due to its speed yet its basic nature prevented it from recognizing intricate patterns within the data thus producing higher prediction errors. The performance of Gradient Boosting was superior yet it encountered limitations when processing extensive datasets with numerous categorical variables.

We shifted towards advanced boosting models during subsequent development stages by selecting XGBoost [2][5] and LightGBM [3]. LightGBM produced better results than all other models when trained on our dataset that contained multiple features and text-derived columns and categorical variables. The model performed best with LightGBM due to its handling of missing data points and its ability to process extensive datasets.

We utilized RandomizedSearchCV [1] to find optimal values for the hyperparameters including number of leaves and learning rate and max depth and regularization terms while validating our model settings through 3-fold cross-validation. The combination of selecting appropriate models and performing parameter optimization resulted in our highest MAE score and excellent data generalization capabilities.

We selected LightGBM because it showed superior performance in validation metrics and offered both fast training times and demonstrated success in solving tabular prediction tasks that resemble our problem.

C. Hyperparameter Tuning

The LightGBM model received optimal predictive performance through careful selection of its parameters. We started with grid search but switched to RandomizedSearchCV because the many parameters combined with high computational expenses made grid search impractical. The method allowed us to explore various parameter combinations while finding the best options at a faster rate because of our large dataset size and restricted computational power.

We conducted tuning operations on multiple essential model parameters.

- **num_leaves** (controls tree complexity): [15, 31, 50, 80]
- **learning_rate** (step size): [0.03, 0.05, 0.1, 0.15]
- **max_depth** (tree depth): [3, 5, 8, -1]
- **min_child_samples** (minimum data in one leaf): [10, 20, 30]
- **subsample** (row sampling): [0.7, 0.8, 1.0]
- **colsample_bytree** (feature sampling): [0.7, 0.9, 1.0]
- **reg_alpha** and **reg_lambda** (regularization): [0, 0.1, 1]

We evaluated each parameter combination through 3-fold cross-validation with Mean Absolute Error (MAE) as the performance metric. The method protected against overfitting by identifying a predictive model which performed well on new data points. The implementation of parameter optimization resulted in lower validation MAE values and improved prediction stability compared to running default settings.

The best parameters discovered through randomized search produced an MAE of 169.20 on the validation data for our final model. Our model achieved substantial accuracy improvement and enhanced robustness because of hyperparameter tuning procedures.

III. RESULTS

The final LightGBM model delivered strong predictive performance. Our validation set produced a Mean Absolute Error (MAE) of \$169.20 that exceeded baseline performance from linear regression and untuned ensemble models. The performance enhancement demonstrates the effectiveness of our method that combined data preprocessing with feature engineering and hyperparameter tuning.

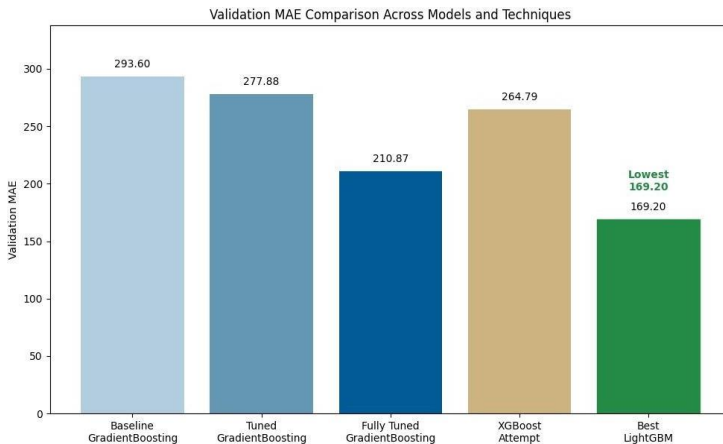


Figure 3: The validation MAE comparison shows different models and feature engineering strategies. Lower is better. The LightGBM pipeline with advanced preprocessing and tuning achieves the best result.

A. Feature Importance

The built-in feature importance metrics of LightGBM identified square_feet as the leading predictive factor together with binned latitude and longitude and number of bedrooms and bathrooms and cityname and state and TF-IDF-derived textual features. [3]

- **square_feet:** The most important predictor in our model was square_feet which indicated that larger apartments usually had higher rental prices.
- **Binned latitude and longitude:** The grouping of these into binned categories enabled the model to detect neighborhood effects that substantially affect rental prices.
- **Number of bedrooms and bathrooms:** The model utilized the information from these features to determine apartment dimensions and functionality.
- **cityname and state:** The model needed geographic indicators such as cityname and state to detect market differences between locations.
- **TF-IDF derived textual features:** The extracted keywords from descriptions and amenities including "pool" "furnished" "pet-friendly" proved to be strong predictors in the model which demonstrated the value of unstructured data.

B. Process Improvements and Key Takeaways

The project underwent repeated improvements in preprocessing and feature engineering methods which produced measurable reductions in the model's Mean Absolute Error. Outlier removal and feature engineering techniques resulted in performance improvements that were measurable. The change to randomized hyperparameter search produced both better model results and shorter computational times. The predictive accuracy could be improved through additional text analysis methods along with economic indicators from outside sources.

IV. DISCUSSION

The project proves that machine learning succeeds at complex real-world problems such as rent prediction through the combination of numerical data with categorical data and textual information. [6] The combination of diverse signals and through feature engineering [4] [7] led to a detailed representation of each apartment listing which proved essential for achieving high model accuracy.

We chose to use latitude and longitude binning instead of using actual coordinates or complex clustering methods for geographic data. The model gained the ability to detect neighborhood patterns through this approach which prevented it from developing strong connections to specific address information. Previous tests using raw coordinates produced less reliable outcomes but binning created a balance between maintaining spatial detail and achieving generalizability.

The TF-IDF vectorization method extracted essential descriptive elements from listings through textual information. The number of TF-IDF features for each text field underwent evaluation to determine the optimal quantity that provided both effective expression and computational efficiency. [8] The results showed how language affects rental price expectations and demonstrated the value of unstructured data for predictive modeling.

We discovered through the process that combining strong preprocessing with continuous experimentation including feature engineering tests and model type assessments produced continuous progress. The RandomizedSearchCV method enabled efficient optimization of hyperparameters and LightGBM handled our large dataset because of its scalability features. [3]

Our scalable modular framework both delivered precise price forecasts and establishes a flexible system which works for ongoing real estate market decision support and analysis.

REFERENCES

- [1] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, and E. Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [5] J. Brownlee, *Machine Learning Mastery with XGBoost*. Machine Learning Mastery, 2021.
- [6] S. Raschka and V. Mirjalili, *Python Machine Learning*, 3rd ed. Birmingham, UK: Packt Publishing, 2020.
- [7] R. Hecht-Nielsen, “Theory of the backpropagation neural network,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 1989.
- [8] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [9] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 51–56.